

Задание 2. Проектирование RTB и интеграции с DSP

1. Проектирование сервиса ставок

1.1 Границы сервиса ставок (bidding Service)

Bidding Service (ранее – часть монолита, в новой архитектуре – отдельный микросервис) является входной точкой для всех RTB-запросов от DSP и Publishers. Его основная задача – координировать процесс аукциона и формировать финальный ответ, но саму логику аукциона он делегирует выделенному Auction Engine.

Что входит в границы Bidding Service

- Приём Bid Request от внешних систем (DSP, SSP) по протоколу OpenRTB (HTTP/2 + JSON) через API Gateway.
- Валидация запроса (обязательные поля, формат, подписи).
- Идентификация пользователя – извлечение user_id из cookie, device ID, IP и т.д.
- Получение пользовательского профиля – запрос к User Profile Service (или чтение из Redis-кеша) для получения сегментов аудитории, frequency caps.
- Поиск подходящих кампаний – запрос к Сервису рекламных кампаний (или чтение из Redis-кеша) для получения всех активных кампаний, релевантных для данного запроса (по гео, устройству, времени, аудиторным сегментам).
- Сбор кандидатов – формирование списка участников аукциона для Auction Engine.
- Вызов Сервису рекламных кампаний (синхронно, gRPC) для определения победителя.
- Запрос баннера – при получении победителя вызывается Creative Service (синхронно, gRPC) для генерации финальной разметки.
- Формирование и отправка Bid Response обратно клиенту.
- Асинхронная публикация событий (клики/показы) в Kafka для дальнейшей обработки.

Что НЕ входит в границы сервиса ставок:

- Логика самого аукциона (правила, взвешивание) – это ответственность Auction Engine.
- Генерация баннера – Creative Service.

- Управление кампаниями и их правилами – Campaign Rules Service (только чтение через кеш/CDC).
- Долгосрочное хранение данных – сервис stateless, все данные получает из кешей и внешних сервисов.

1.2 API сервиса

- Внешний API (через API Gateway)
- Протокол: gRPC поверх HTTP/2 (для низкой задержки и бинарной сериализации).
- Формат сообщений: Protocol Buffers.
- Основные API (ориентировочный вариант)
 - BidRequest → BidResponse (основной RTB-эндпоинт)
Назначение: аукцион за показ.
 - Входящие поля: ad_unit_id, user_id (или cookie/device_id), site/app, geo, device_type, context, timestamp.
 - Исходящие поля: winner_campaign_id, creative_url, tracking_pixels, price_cpm, ttl_ms, debug_info (для отладки).
 - HealthCheck → HealthStatus Для Kubernetes readiness/liveness. Возвращает статус Redis, PostgreSQL replica, Kafka producer.
 - Экспорт внутренних метрик (latency, error rates, cache hit/miss) для систем мониторинга (Prometheus.)

1.3 Зависимости от других сервисов

Зависимость	Тип связи	Протокол	Цель
Сервис рекламных кампаний	Синхронная критический вызов	gRPC	Получение правил, лимитов, статусов кампаний.
Сервис выдачи банера	Синхронная критический вызов	gRPC	Генерация баннера для победителя.
Redis	Синхронно	RESP	Горячие данные: правила, лимиты, профили пользователей, счётчики частоты показов.
PostgreSQL (readreplica)	Синхронно, только SELECT по ключам	PG protocol	Источник истины для правил и лимитов, если нет в кэше.

Kafka	Асинхронно (fireandforget)	Kafka protocol	Публикация событий events.raw (показ, клик, viewability).
API Gateway	Синхронно	gRPC/HTTP	Маршрутизация, rate limiting, auth, трейсинг.

Критический путь (должен укладываться в <80 мс) включает:

- чтение из Redis;
- вызов сервис рекламных кампаний;
- вызов сервис выдачи.

Важные паттерны для зависимостей:

- Cacheaside: сначала Redis, если нет — PostgreSQL, затем обновление кэша.
- Circuit Breaker: для защиты от каскадных отказов при проблемах с Kafka или PostgreSQL.
- Идемпотентность: события в Kafka должны быть идемпотентными, чтобы повторная отправка не ломала учёт.

Модель данных сервиса ставок

У сервиса ставок нет собственной транзакционной БД. Он использует:

- Redis — для горячих данных (в виде структур: hashes, sorted sets, bitmaps).
- PostgreSQL readreplica — как источник истины (только SELECT).

2. Проектирование взаимодействия

2.1 Протоколы и паттерны взаимодействия в целевой архитектуре AdScale

2.1.1 Внешний протокол интеграции с DSP

Протокол: HTTP/2 + JSON (OpenRTB 2.5+)

Обоснование:

- OpenRTB является индустриальным стандартом для Real-Time Bidding, поддерживается всеми крупными DSP и SSP.
- Использование JSON упрощает отладку и интеграцию с партнёрами, не требует генерации клиентского кода.
- HTTP/2 позволяет мультиплексировать запросы и уменьшить накладные расходы.
- При необходимости для продвинутых партнёров можно предоставить gRPC + Protobuf как альтернативный эндпоинт, но базовым остаётся OpenRTB.

2.1.2 Внутренние протоколы взаимодействия между микросервисами

Сервис ставок (критический путь, $\leq 80\text{--}100$ мс P95) → gRPC

Протокол: gRPC поверх HTTP/2, сообщения в Protocol Buffers.

- Request/Response (синхронно) для самого аукциона.
- Circuit Breaker на вызовах зависимостей (чтобы не «положить» аукцион, если Redis или PostgreSQL временно тормозит).
- Cache-aside (сначала Redis, потом PostgreSQL, затем обновление кэша).
- Retry с экспоненциальной задержкой для кратковременных сбоев.

Обоснование:

- Низкая задержка: Бинарная сериализация Protobuf значительно быстрее JSON, а HTTP/2 позволяет использовать постоянные соединения и мультиплексирование;
- Строгие контракты: .proto-файлы обеспечивают типизацию и генерацию кода на многих языках, упрощая поддержку;
- Управление тайм-аутами и отменами: gRPC поддерживает контекстные тайм-ауты и отмену запросов, критично для соблюдения SLA;

2.1.3. Асинхронное взаимодействие (событийно-ориентированное)

Протокол: Apache Kafka

Обоснование:

- Разделение критического пути: Отправка событий в Kafka производится асинхронно и не блокирует ответ аукциона.
- Буферизация и сглаживание пиков: Kafka выступает как буфер, позволяя потребителям (статистика, биллинг) обрабатывать данные в своём темпе.
- Гарантии доставки: Поддержка at-least-once / exactly-once семантики, возможность replay при сбоях.
- Масштабируемость: Независимые группы потребителей позволяют горизонтально масштабировать каждый сервис-потребитель
- Устойчивость к отказам: Если потребитель временно недоступен, сообщения не теряются и могут быть обработаны позже.

Формат: Avro + Schema Registry — чтобы контролировать эволюцию схем и избежать «сломанных» сообщений при изменениях контрактов.

2.2 API Gateway

API Gateway обязателен в целевой архитектуре

Обоснование:

- Единая точка входа для всех внешних запросов (DSP, рекламодатели, внутренние системы).
- Маршрутизация: Направляет запросы к соответствующим микросервисам (/rtb/bid → Bidding Ingress, /api/campaigns → Campaign Rules и т.д.).
- Безопасность: TLS-терминация, аутентификация (JWT/API Keys), rate limiting, защита от DDoS.
- Управление трафиком: Canary-релизы, A/B-тестирование, откаты через динамическую маршрутизацию.
- Наблюдаемость: Сбор метрик, логирование, распределённая трассировка (через интеграцию с OpenTelemetry).
- В перспективе - Geo-DNS: Направление запросов в ближайший регион для снижения задержек.

2.3 Итог

Взаимодействие	Протокол	Обоснование
Внешнее (DSP → AdScale)	OpenRTB (HTTP/2 + JSON)	Индустриальный стандарт, совместимость, простота.
Внутреннее синхронное (RTB-путь)	gRPC	Минимальная задержка, строгие контракты, поддержка потоков и тайм-аутов.
Внутреннее синхронное (UI, администрирование)	REST (HTTP/JSON)	Простота, удобство для фронтенда, OpenAPI-документация.
Внутреннее асинхронное (события)	Kafka	Буферизация, надежность, replay, разделение критического пути.

3. API Gateway для DSP-интеграции

Интеграция с новой DSP-площадкой требует надёжного, безопасного и производительного API-слоя, который обеспечит приём до 18 000 дополнительных RPS (первоначально) и соблюдение жёсткого SLA по latency (<80 мс). Основой этого слоя является **API Gateway (реализация возможна с помощью NGINX или Envoy Proxy)**, который выступает единой точкой входа, а также реализует ряд функций безопасности, отказоустойчивости и управления запросами.

3.1 . Маршрутизация запросов на ставки

- Все внешние RTB-запросы поступают на единый эндпоинт (API Gateway);
- API Gateway маршрутизирует запросы в сервис ставок;
- Для новой DSP-площадки может потребоваться специфическая обработка (например, дополнительная валидация, трансляция полей OpenRTB).

3.2 Ограничение частоты запросов (Rate Limiting)

Для защиты системы от перегрузки и соблюдения SLA, необходимо ограничить количество запросов от каждого DSP-партнёра.

Реализация:

- Используем функцию API GW Rate - Limit Filter;
- Настраиваем лимиты как для всей платформы (глобальный) и для DSP (по X-DSP-Id);
- Для нового DSP: начальный лимит 20 процентов от максимального RPS (с возможностью повышения);
- При превышении лимита Gateway возвращает HTTP 429 (Too Many Requests) без передачи запроса в бэкенд.

Дополнительно можно применить сглаживание пиков запросов / трафика (token bucket).

3.3. Аутентификация

Для новой DSP-площадки выбираем **mTLS (Mutual TLS)** как основной метод аутентификации, так как он обеспечивает высокий уровень безопасности и не добавляет заметной задержки.

Обоснование:

- mTLS проверяет как подлинность клиента (сертификат), так и сервера, что исключает подделку запросов.
- Не требует передачи секретов в каждом запросе (в отличие от API-ключей или JWT, которые нужно валидировать и парсить).
- Вписывается в инфраструктуру с использованием API GW, который поддерживает mTLS из коробки (через SDS для сертификатов).

3.4. Паттерн автоматического размыкателя (Circuit Breaker)

Circuit Breaker применяется как к вызовам внутренних зависимостей на уровне API-шлюза (API Gateway) так и со стороны внешнего партнёра (например, если DSP начинает отправлять слишком много запросов, превышая лимиты, или при аномальном поведении). Таким образом, применяя данный паттерн мы не усиливаем каскадные отказы и снижаем вероятность «положить» сервис ставок.

3.5. Мониторинг времени отклика

Для соблюдения SLA ($P95 < 80$ мс) критично измерять и визуализировать время ответа на каждом этапе.

- API GW экспортирует встроенную статистику
- Возможно использовать стандарт OpenTelemetry для распределённой трассировки каждого запроса (через заголовки traceparent).
- В сервисы ставок и кампаний также добавляем метрики (время на каждый вызов) и публикуем их.

Настройка связки Prometheus / Grafana

Алерты:

- Настраиваем алерты в **Prometheus** на превышение $P95 \text{ latency} > 70$ ms для кластера Bidding Ingress (запас для учёта времени на обработку внутри).
- Отслеживаем процент ошибок (5xx) и количество срабатываний Rate Limit / Circuit Breaker.

Дашборды в Grafana отображают:

- Latency по перцентилям (P50, P95, P99) для всего API-слоя и по каждому DSP-партнёру.
- Тренды ошибок и использования лимитов.
- Трассировка отдельных запросов для глубинного анализа задержек.

3.6 Итог

Таким образом, API-слой обеспечивает:

- **Безопасность** через mTLS.
- **Защиту от перегрузок** через rate limiting и circuit breaker.
- **Гибкость** маршрутизации для постепенного ввода нового DSP.
- **Наблюдаемость** для подтверждения соответствия SLA.

Все компоненты конфигурируются декларативно через Kubernetes + Envoy и легко масштабируются при росте нагрузки.

4. Паттерны надёжности

Надёжность — ключевой фактор для соответствия SLA 99.9% и соблюдения жёстких требований DSP по latency. В архитектуре AdScale применяются следующие паттерны и стратегии.

4.1 Circuit Breaker для вызовов зависимых сервисов

Применяется на всех синхронных зависимостях:

- Closed — нормальная работа, запросы идут.
- Open — после серии ошибок (например, 5 ошибок подряд или > 90 % ошибок за период времени) сервис перестаёт делать запросы и сразу возвращает заранее заготовленный ответ (дефолт/кэш).
- Half-Open — периодически пробует 1–2 запроса; если успешно — возвращается в Closed, если нет — обратно в Open.

Также необходимо использовать **паттерн Retry** (Повторные попытки) с экспоненциальной задержкой и джиттером. Используется в случае возникновения временных (сетевых) ошибок, таймаутов, 503 и пр.

Circuit Breaker для сервиса ставок (низкий latency):

- максимум 2–3 попытки;
- экспоненциальная задержка: 50 мс → 100 мс → 200 мс;
- джиттер (random) $\pm 20\%$ к задержке, чтобы избежать «грозовой атаки» (thundering herd);
- общий бюджет времени на все повторы : $\leq 20\text{--}30$ мс, чтобы не превысить SLA 80–100 мс.

4.2 Идемпотентность финансовых операций

Финансы нельзя «начислить дважды» из-за повторов или дублей сообщений.

Для реализации транзакционности в финансовых операциях используется паттерн SAGA. Каждое финансовое событие (например, списание за показ) содержит уникальный идентификатор `transaction_id` (генерируется на основе `auction_id + impression_id`).

- В таблице транзакций создаётся уникальное ограничение на `transaction_id`.

- Перед выполнением списания сервис проверяет наличие записи с таким ID в БД:
- Если запись существует - игнорирует (идемпотентный ответ).
- Если нет - выполняет списание и вставляет запись в той же транзакции.

Для дополнительной защиты, если БД временно недоступна, сервис может временно сохранять обработанные ID в Redis-сети (с TTL, например, 24 часа) для быстрой проверки.

4.3 Резервные стратегии (Fallback / Graceful Degradation)

При недоступности зависимых сервисов или превышении тайм-аутов, система должна продолжать работать, пусть и с пониженной точностью.

Дефолтные ставки (Fallback)

Если Redis/PostgreSQL недоступны, Bidding Service может:

- вернуть «нет рекламы» (safest);
- отдать дефолтную ставку по паре ad_unit + geo из локального конфига или из Redis с долгим TTL;
- применить упрощённые правила: без таргетинга по устройствам/категориям, только гео и статус кампании.

Стратегия деградации функционала (Degradation)

- Полная потеря Redis: использовать только PostgreSQL replica (медленнее, но надёжно).
- Потеря PostgreSQL replica: работать только по кэшу или с мастер данными.
- Потеря Kafka: события можно временно не писать (или писать в локальный буфер/лог), но ставки продолжать отдавать.